Lecture L02

*Blind review — please score this lecture on row **L02** in the Excel form (1–10 per dimension).*

Introduction to Backpropagation in Neural Networks

Imagine you're trying to teach a child to throw a ball into a basket. At first, they might throw it too short or too long, but with each attempt, you provide feedback on how to adjust their throw. This process of adjusting based on feedback is similar to how neural networks learn, and it's made possible by an algorithm called backpropagation. By the end of this lecture, we will be able to explain the motivation behind backpropagation and its role in training neural networks, apply the backpropagation algorithm to compute the gradient of a cost function with respect to a neural network's parameters, analyze the design decisions involved in training a feedforward network using backpropagation, identify common pitfalls and limitations of the backpropagation algorithm, and evaluate the historical development and significance of backpropagation in deep learning.

To begin, consider a simple example. Suppose we have a neural network that accepts an input x and produces an output $\hat{y}$. We want to train this network to minimize the difference between $\hat{y}$ and the true output y. To do this, we need to compute the gradient of the cost function with respect to the network's parameters. This is where backpropagation comes in. Try to think of a scenario where you have to adjust multiple factors to achieve a desired outcome, like adjusting the mirrors in a periscope to get a clear view. How would you approach this problem, and what does it have to do with backpropagation? This concept is similar to the idea of "sensitivity analysis" in operations research, where we analyze how changes in input parameters affect the output of a system.

Motivation for Backpropagation

When using a feedforward neural network, information flows forward through the network, a process known as forward propagation. However, during training, we need to compute the gradient of the cost with respect to the model's parameters. Computing an analytical expression for the gradient is straightforward but can be computationally expensive. Backpropagation addresses this issue by efficiently computing the gradient using a table-filling algorithm that stores intermediate results. A common misconception is that backpropagation is only used for training neural networks, but it can also be used for other optimization problems. For instance, consider a manufacturing process where you need to optimize the production line to minimize costs. How might backpropagation be applied in this context? This is similar to the concept of "just-in-time" manufacturing, where production is optimized to meet demand in real-time.

The main advantage of using backpropagation over computing the gradient analytically is its efficiency. Backpropagation avoids repeating common subexpressions by storing intermediate results, allowing for faster computation of the gradient. To illustrate this, think of a recipe that requires you to make a sauce, which in turn requires you to make a stock. If you had to start from scratch each time you wanted to make the sauce, it would be inefficient. Instead, you can store the stock and use it whenever you need it, much like backpropagation stores intermediate results. This concept is also related to the idea of "memoization" in computer science, where intermediate results are stored to avoid redundant computation.

The Backpropagation Algorithm

The backpropagation algorithm can be viewed as a dynamic programming approach that avoids repeating common subexpressions by storing intermediate results $\frac{\partial u}{\partial x}$ in a table. This strategy allows for efficient computation of the gradient. To illustrate this, consider a simple example. Suppose we have a neural network with two inputs, x_1 and x_2, and one output, $\hat{y}$. We want to compute the gradient of the cost

function with respect to the weights w_1 and w_2. Using backpropagation, we can compute the gradient as follows:

$$\frac{\partial C}{\partial w_1} = \frac{\partial C}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w_1}$$

where z is the output of the hidden layer. Now, try to derive this equation on your own, thinking about how the gradient of the cost function with respect to each weight is computed. This is similar to the concept of "automatic differentiation" in numerical analysis, where derivatives are computed automatically using the chain rule.

Example: Back-Propagation for MLP Training

As an example, consider a simple multilayer perceptron (MLP) with a single hidden layer. To train this model, we use minibatch stochastic gradient descent, where the back-propagation algorithm is used to compute the gradient of the cost on a single minibatch. The process involves forward propagation to compute the output, followed by backpropagation to compute the gradient of the cost with respect to the model's parameters.

The steps involved in training the MLP are as follows: - Forward propagation: Compute the output of the hidden layer, $z = \sigma(w_1 \cdot x_1 + w_2 \cdot x_2)$, where σ is the activation function. - Backpropagation: Compute the gradient of the cost function with respect to the weights, $\frac{\partial C}{\partial w_1} = \frac{\partial C}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w_1}$.

To derive the backpropagation equations for a neural network with multiple hidden layers, we can apply the chain rule recursively, computing the gradient of the cost function with respect to each layer's weights and biases. Can you think of a scenario where you have to adjust the settings on a camera to get the perfect shot? How does this relate to the process of backpropagation in neural networks? This is similar to the concept of "optical flow" in computer vision, where the motion of objects is tracked by adjusting the camera settings.

Case Study: Image Classification using Backpropagation

Consider a real-world application of backpropagation in image classification. Suppose we want to train a convolutional neural network (CNN) to classify images into different categories, such as dogs, cats, and birds. We can use backpropagation to compute the gradient of the cost function with respect to the model's parameters, which includes the weights and biases of the convolutional and fully connected layers. The backpropagation algorithm is applied recursively, starting from the output layer and moving backwards to the input layer, computing the gradient of the cost function with respect to each layer's parameters.

For example, let's consider a CNN with two convolutional layers and two fully connected layers. The backpropagation algorithm would compute the gradient of the cost function with respect to the weights and biases of each layer, using the chain rule to recursively apply the gradient computation. The gradient of the cost function with respect to the weights of the first convolutional layer would be computed as follows:

$$\frac{\partial C}{\partial w_{conv1}} = \frac{\partial C}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_{fc2}} \cdot \frac{\partial z_{fc2}}{\partial z_{conv2}} \cdot \frac{\partial z_{conv2}}{\partial z_{conv1}} \cdot \frac{\partial z_{conv1}}{\partial w_{conv1}}$$

where z_{conv1} and z_{conv2} are the outputs of the first and second convolutional layers, respectively, and z_{fc2} is the output of the second fully connected layer.

Case Study: Natural Language Processing using Backpropagation

Another example of backpropagation in a real-world application is natural language processing (NLP). Suppose we want to train a recurrent neural network (RNN) to predict the next word in a sentence, given the context of the previous words. We can use backpropagation to compute the gradient of the cost function with respect to the model's parameters, which includes the weights and biases of the recurrent and output layers. The backpropagation algorithm is applied recursively, starting from the output layer and moving backwards to the input layer, computing the gradient of the cost function with respect to each layer's parameters.

For example, let's consider an RNN with two recurrent layers and one output layer. The backpropagation algorithm would compute the gradient of the cost function with respect to the weights and biases of each layer, using the chain rule to recursively apply the gradient computation. The gradient of the cost function with respect to the weights of the first recurrent layer would be computed as follows:

$$\frac{\partial C}{\partial w_{rnn1}} = \frac{\partial C}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_{rnn2}} \cdot \frac{\partial z_{rnn2}}{\partial z_{rnn1}} \cdot \frac{\partial z_{rnn1}}{\partial w_{rnn1}}$$

where z_{rnn1} and z_{rnn2} are the outputs of the first and second recurrent layers, respectively.

Design Decisions for Feedforward Networks

Training a feedforward network requires making several design decisions, including choosing the optimizer, the cost function, and the form of the output units. Additionally, we need to choose the activation functions for the hidden layers and design the architecture of the network, including the number of layers, their connections, and the number of units in each layer. The choice of optimizer is critical, as some optimizers may not be suitable for the problem at hand. For example, suppose we want to train a neural network to classify images. We need to choose an optimizer that can handle the large number of parameters in the network. A good choice would be stochastic gradient descent with momentum. Now, consider a scenario where you're trying to optimize a complex system, like a traffic flow. How would you approach this problem, and what design decisions would you need to make? This is similar to the concept of "traffic signal control" in transportation engineering, where the flow of traffic is optimized by adjusting the timing of traffic signals.

Limitations and Common Pitfalls

While backpropagation is a powerful algorithm, it has some limitations. For instance, it can be computationally expensive for very large networks, and the choice of hyperparameters, such as the learning rate, can significantly affect the convergence of the algorithm. Moreover, backpropagation can be sensitive to the initialization of the model's parameters and may converge to a local minimum instead of the global minimum. Backpropagation does not always converge to the global minimum, and this is a common misconception. Can you think of a scenario where you're trying to find the shortest path between two points, but you get stuck in a local minimum? How does this relate to the limitations of backpropagation? This is similar to the concept of "getting stuck in a local optimum" in optimization theory, where the algorithm converges to a local minimum instead of the global minimum.

To mitigate these limitations, we can use various techniques, such as regularization, early stopping, and batch normalization. Regularization techniques, such as L1 and L2 regularization, can help prevent overfitting by adding a penalty term to the cost function. Early stopping can help prevent overfitting by stopping the training process when the model's performance on the validation set starts to degrade. Batch normalization can help stabilize the training process by normalizing the inputs to each layer. By

using these techniques, we can improve the performance of the backpropagation algorithm and overcome its limitations.